

TYPISCHE CODE-SCHNIPSEL ZUM BÖRSEN-OSZILLOGRAPHEN :

1]BÖRSENTICKER-LISTE DEFINIEREN

```
ticker_df = pd.read_csv('Ticker-Liste_alt.csv')
akt_ticker = ticker_df.iloc[akt_index]
lade_ticker_daten(f"{akt_ticker['Num']:02d}",akt_ticker['Nam'],akt_ticker['Tik']
```

2]BÖRSENTICKER LADEN

```
df_live = yf.Ticker(ticker_str).history(start=ANF_DAT,end=END_DAT)[["Close"]]
df_live.index = df_live.index.tz_localize(None)
df_csv = df_live.rename(columns={"Close": "Price"}).dropna().reset_index()
df_csv['Date'] = pd.to_datetime(df_csv['Date'])
df_csv.to_csv(csv_nome, index=False)
df = df_csv.sort_values('Date').reset_index(drop=True)
```

3]TASTEN MIT MATPLOTLIB DEFINIEREN

```
ticker_klavier = []
schalter_handles = []
anzahl_ticker = len(ticker_df)
for i in range(anzahl_ticker):
    zeil = ticker_df.iloc[i]
    num_str = f"{zeil['Num']:02d}"
    btn_label = f"{num_str} {zeil['Nam']}"
    y_pos = 0.97 - (i * 0.033)
    ax_tick = fig.add_axes([0.01, y_pos, 0.08, 0.028])
    btn_color = '#b0c4de' if i == akt_index else '#e1e1e1'
    btn = Button(ax_tick, btn_label, color=btn_color)
    btn.label.set_fontsize(10)
    ticker_klavier.append((btn, num_str, zeil['Nam'], zeil['Tik'], i))
schalter_handles.append(btn)
```

SCHRITT-FÜR-SCHRITT ÜBERSETZUNG:

1]BÖRSENTICKER-LISTE DEFINIEREN UND ZUM LADEN NUTZEN

```
ticker_df = pd.read_csv('Ticker-Liste_alt.csv')
alte Ticker-Liste aus CSV-Datei in Pandas-DataFrame laden.
akt_ticker = ticker_df.iloc[akt_index]
Daten aktueller Ticker anhand Zeilennummer akt_index ausgelesen.
lade_ticker_daten(f"{akt_ticker['Num']:02d}",akt_ticker['Nam'],akt_ticker['Tik']
Funktionsaufruf: aktuellen Ticker per Nummer(zweistellig),Name,Symbol laden.
```

2]KONKRETE BÖRSENTICKER-DATEN LADEN UND ZEITGEMÄSS AUFBEREITEN

```
df_live = yf.Ticker(ticker_str).history(start=ANF_DAT,end=END_DAT)[["Close"]]
Aktiendaten für ticker_str im Zeitraum laden, nur die Schlusskurse (Close) behalten.
df_live.index = df_live.index.tz_localize(None)
Zeitzoneinfo aus dem Datums-Index entfernen, gegen Probleme bei fehlender Zeitzone.
df_csv = df_live.rename(columns={"Close": "Price"}).dropna().reset_index()
Spalte Close in Price umbenannt, fehlende Werte löschen, Datum wird normale Spalte
df_csv['Date'] = pd.to_datetime(df_csv['Date'])
Werte in Spalte Date in ein standardisiertes Datumsformat umwandeln.
df_csv.to_csv(csv_nome, index=False)
Daten werden ohne Zeilenindex als CSV-Datei csv_nome auf Festplatte gespeichert.
df = df_csv.sort_values('Date').reset_index(drop=True)
Daten chronologisch nach Datum sortieren, Zeilennummern neu durchnummerieren.
```

3]BÖRSENTICKER-LISTE PER SCHLEIFE ALS TASTATUR AM LINKEN BILDRAND AUFBAUEN

a]SCHLEIFEN-VORBEREITUNG

```
ticker_klavier = []
```

Leere Liste um Steuerungselemente und Daten der Ticker-Tasten zu speichern.

```
schalter_handles = []
```

leere Liste um Button-Objekte im Speicher halten / kein autom. Löschen

```
anzahl_ticker = len(ticker_df)
```

Gesamtzahl der gefundenen Ticker ermitteln und speichern

```
for i in range(anzahl_ticker):
```

Schleife starten um nacheinander jeden Ticker aus der Liste zu verarbeiten

```
zeil = ticker_df.iloc[i]
```

Daten der aktuellen Ticker-Zeile für aktuellen Schleifendurchlauf geladen.

b]TASTEN-BESCHRIFTUNG , POSITIONIERUNG UND EINFÄRBUNG

```
num_str = f"{zeil['Num']:02d}"
```

Ticker-Nummer in Text umwandeln auf zwei Stellen mit führenden Null aufgefüllt

```
btn_label = f"{num_str} {zeil['Nam']}"
```

Text für Tasten-Beschriftung aus formatierter Nummer und Namen zusammengesetzt

```
y_pos = 0.97 - (i * 0.033)
```

vertikale Position aktuelle Taste so berechnen dass alle Tasten untereinander

```
ax_tick = fig.add_axes([0.01, y_pos, 0.08, 0.028])
```

Auf Matplotlib-Grafik neuen Bereich für die Platzierung der Taste definieren

```
btn_color = '#b0c4de' if i == akt_index else '#e1e1e1'
```

Farbe der Taste Hellblau wenn aktiv , andernfalls hellgrau gefärbt

c]TASTE MIT INTERAKTIVER_MATPLOTLIB-WIDGET FUNKTION AUSSTATTEN

```
btn = Button(ax_tick, btn_label, color=btn_color)
```

Interaktives Button-Objekt mit berechneter Position,Beschriftung und Farbe

```
btn.label.set_fontsize(10)
```

Text-Schriftgröße aktuelle Taste auf 10 Punkte eingestellt

```
ticker_klavier.append((btn, num_str, zeil['Nam'], zeil['Tik'], i))
```

Button mit relevanten Ticker-Informationen als Paket in Liste speichern

d]TASTE INKLUSIVE FUNKTIONALITÄT UM SIE WÄHREND DEM PROGRAMMLAUF AKTIV ZU HALTEN

```
schalter_handles.append(btn)
```

Button Schutz-Liste ablegen,damit Python ihn nicht löscht.

WARUM DIESE MAßNAHME ?

Im Gegensatz zu einem EXE-Programm ist ein Python-Programm keine statische Speicherbelegung.Es wird laufend ausgelesen (**interpretiert**) und die Folge der Programzustände mit den dadurch adressierten Variablen belegt möglichst gekapselt Speicherplatz und baut ihn wieder ab. Eine ausserhalb dieser Zyklen definierte Liste besteht aus **Zeigern**, also relativen Adressen, die mit einander zusammenhängen. Wenn dieses Konstrukt aus Zeigern **onRuntime** erhalten bleibt, werden die Zeiger die „**Lebensversicherung**“ der damit erfassten Objekte und die entsprechende „Adressenwolke“ stellt eine feste Speicherbelegung dar.

Die Liste **schalter_handles** ist so lange "aktiv" wie sie selbst im Speicher existiert.Wenn **schalter_handles** auf höherer Ebene (**global**) definiert ist, während der Garbage Collector im Hintergrund Verweise zählt (sog.Reference Counting) besitzt der Button **mindestens einen Zeiger** und bleibt somit aktiv.

Da der Notebook-Betrieb statisch ausgelegt ist und eine Folge von Bildern (PNG's) darstellt, muß der interaktive Börsen-Oszillograph im Fall von Widgets darauf achten, daß seine Bilderfolge wie ein „**DAUMENKINO**“ möglichst ohne jede Unterbrechungen(**Interrupts**) abgespult wird. Er ist eine Dauerschleife (deFacto Animation durch ständiges Schaubild-Update) die erst mit dem Schliessen aller (bzw möglichst nur **EINES**) Fensters endet, was per plt.show() angekündigt ist.

Die Aktivität der Widgets kostet Rechenzeit „in Präsenz“ und wird wie folgt streng getaktet .Zu diesem Zweck werden (**wieder über eine fest verzeigte Liste**) alle Widgets verkettet und während dem Zeichen-Takt der Dauerschleife synchron wie eine **Christbaum-Beleuchtung AUS- und wieder EIN**-geschaltet. Im Betrieb kann man das besonders an den Textboxen der Datums-Eingaben sowie an der „**Hover-Farbwechsel-Schleppe**“ des Ticker-Klaviers beobachten. Der Magic-Befehl **%Matplotlib** widget macht dieses Interrupt-Tuning möglich. Ohne ihn ist das „Daumenkino“ eingefroren und Python somit keine Interaktive Oberfläche.

Weil Buttons Pixel-sensitiv überwacht werden, also rechnerisch aufwändig auf die Nähe des Mauszeigers reagieren und dadurch „Zeit verschwenden“, stellt Matplotlib ihnen den **boolschen Schalter „eventson“** zur Verfügung - **Textboxen ausgenommen**, deren Latenz persistiert. Es gilt daher, den senkrechten „Edier-Cursor“ im Auge zu behalten, und erst nach „Updaten“ seiner Position im Takt eine Änderung des angezeigten Textes fortzusetzen.

Eine Textbox reagiert NICHT pixelorientiert sondern mit dem Cursor-Updating und (**NICHT ZU VERGESSEN!**) benötigt den dezidierten Abschluss durch die **ENTER** Taste. Erst dieses **ENTER** führt durch **SUBMIT**-Verarbeitung zu einer **Eingabe** .

ERSTER SCHRITT VOR DER SCHLEIFE : BÜNDELUNG IN EINER LISTE

Ganz am Ende des Scripts, kurz vor dem ERSTEN Schleifen-Aufruf :

```
# (widget_event-Liste):
```

```
all_widget_events = [ slider_von,
                      slider_bis,
                      text_box_start,
                      text_box_end,
                      btn_clr,
                      btn_online,
                      btn_diag1,
                      btn_diag2,
                      btn_diag3,
                      btn_diag4]
```

```
# und danach noch das Klavier (die 26 Schalter) obendrauf:
```

```
all_widget_events.extend(ticker_klavier)
```

ZWEITER SCHRITT IN DER SCHLEIFE : CHRISTBAUM-AUS und CHRISTBAUM-AN !

AM SCHLEIFEN-ANFANG:

```
for obj in all_widget_events:
    if hasattr(obj, 'eventson'):
        obj.eventson = False
    elif isinstance(obj, tuple):
        for sub_obj in obj:
            if hasattr(sub_obj, 'eventson'):
                sub_obj.eventson = False
```

AM SCHLEIFEN-ENDE:

```
for obj in all_widget_events:
    if hasattr(obj, 'eventson'):
        obj.eventson = True
    elif isinstance(obj, tuple):
        for sub_obj in obj:
            if hasattr(sub_obj, 'eventson'):
                sub_obj.eventson = True
```